LAPORAN PRAKTIKUM

STRUKTUR DATA

“ STACK ARRAY ”

disusun Oleh:

FILZI JELILA INDA ROBBANI

(2511533019)

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan mengenai penggunaan struktur data Stack dengan Array pada ADT.

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
BAB I PENDAHULUAN **4**
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 5
BAB II PEMBAHASAN **6**
2.1 Langkah Kerja Praktikum 6
2.1.1 Membuat kelas pertama (stackArray) 6
2.1.2 Membuat kelas kedua (stackArrayDriver) 8
2.1.3 Membuat program ketiga (StackPostfix) 9
2.1.4 Membuat program keempat (NilaiMaksimum) 11
2.1.5 Membuat program keempat (SiswaStack) 13
2.2 Tugas Praktikum 16
2.2.1 Deskripsi Tugas 16
2.2.2 Membuat Kelas ADT Website 16
2.2.3 Membuat Driver (Browser_2511533019) 17
BAB III PENUTUP **12**
3.1 Kesimpulan 12
DAFTAR PUSTAKA **13**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, telah dipelajari mengenai Abstract Data Type (ADT) serta pengimplementasian struktur data ArrayList dalam bahasa pemrograman Java. ArrayList merupakan salah satu implementasi dari struktur data, yang memungkinkan penyimpanan data dengan ukuran yang fleksibel serta menyediakan berbagai operasi seperti penambahan, penghapusan, pencarian, dan pengubahan data.

Selanjutnya pada praktikum kali ini mempelajari pengimplementasian struktur data lainnya, yaitu **stack**. Stack merupakan struktur data yang memiliki prinsip *Last In First Out* (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Struktur data ini banyak digunakan dalam berbagai aplikasi, seperti pengelolaan memori, pemanggilan fungsi, serta pengolahan ekspresi matematika.

Dalam praktikum ini, ArrayList digunakan dalam beberapa kasus. Hal ini bertujuan agar mahasiswa dapat memahami keterkaitan antara konsep ADT dan Struktur Data yang telah dipelajari sebelumnya dengan implementasi nyatanya dalam pemrograman.

Dengan demikian, praktikum ini menjadi lanjutan yang penting untuk memperkuat pemahaman mahasiswa dalam mengelola data secara dinamis, terstruktur, dan berbasis objek.

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

a. Memahami konsep dasar struktur data **stack** beserta prinsip kerjanya yaitu *Last In First Out* (LIFO).

b. Mampu mengimplementasikan struktur data stack dalam bahasa pemrograman Java.
c. Memahami dan mengimplementasikan proses evaluasi ekspresi postfix menggunakan stack.
d. Mampu melakukan pengolahan data untuk mencari nilai maksimum dari sekumpulan data.
e. Meningkatkan kemampuan dalam menyusun algoritma serta mengimplementasikannya dalam penyelesaian masalah menggunakan struktur data yang tepat.

1.3 Manfaat

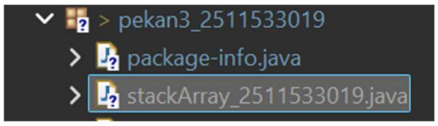
a. Menambah pemahaman mengenai konsep dan penerapan struktur data **stack** dalam pemrograman.
b. Melatih kemampuan dalam mengimplementasikan evaluasi ekspresi postfix menggunakan stack.
c. Meningkatkan keterampilan dalam menyusun algoritma untuk menyelesaikan suatu permasalahan.
d. Membantu mahasiswa memahami penggunaan struktur data dalam pengolahan data secara efisien dan terstruktur.
e. Mengembangkan kemampuan pemrograman dalam bahasa Java, khususnya dalam penerapan struktur data dan logika algoritma.

BAB II
PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (stackArray)

1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama pekan3 terus pilih “New”, pilih class. Buat nama class “stackArray_2511533019”.



2) Lalu, masukkan syntax seperti berikut.

```
1)  package pekan3_2511533019;
2)
3)  public class stackArray_2511533019 {
4)      static final int MAX_3019 = 10000;
5)      int top_3019;
6)      int a_3019[] = new int[MAX_3019];
7)      boolean isEmpty_3019()
8)      {
9)              return (top_3019 < 0);
10)     }
11)     stackArray_2511533019()
12)     {
13)             top_3019 =-1;
14)     }
15)     boolean push_3019(int x)
16)     {
17)             if (top_3019 >= (MAX_3019 -1)) {
18)                     System.out.println("Stack Overflow");
19)                     return false;
20)             }else {
21)                     a_3019[++top_3019]=x;
22)                     System.out.println(x + " dimasukkan dalam
    stack");
23)                     return true;
24)             }
25)     }
26)     int pop_3019()
27)     {
28)             if (top_3019<0) {
29)                     System.out.println("Stack Underflow");
30)                     return 0;
31)             }else {
32)                     int x= a_3019[top_3019--];
33)                     return x;
34)             }
35)     }
36)     int peek_3019()
37)     {
38)             if (top_3019 < 0) {
39)                     System.out.println("Stack underflow");
40)                     return 0;
41)             }else {
42)                     int x = a_3019[top_3019];
```

```
                        return x;
                }
        }
        void print_3019() {
                for(int i = top_3019;i>-1;i--) {
                        System.out.print(" " + a_3019[i]);
                }
        }
}
```

Program pertama yaitu *stackArray_2511533019* bertujuan untuk memahami penggunaan dasar stack pada Java. Pada program ini, terdapat variabel MAX_3019 yang berfungsi sebagai kapasitas maksimum stack, yaitu sebanyak 10000 elemen, serta array a_3019 yang digunakan untuk menyimpan data. Variabel top_3019 digunakan sebagai penunjuk posisi elemen teratas dalam stack.

Program ini memiliki beberapa method utama, yaitu isEmpty() di program saya isEmpty()_3019 yang digunakan untuk mengecek apakah stack dalam keadaan kosong dengan cara memeriksa apakah nilai top_3019 kurang dari nol.

Konstruktor stackArray_2511533019() menginisialisasi stack dengan nilai awal top_3019 = -1 yang menandakan bahwa stack masih kosong.

Method push() di program saya tulis push_3019(int x) berfungsi untuk menambahkan elemen ke dalam stack, dengan terlebih dahulu memeriksa apakah stack sudah penuh (*overflow*). Jika belum penuh, maka nilai top_3019 akan dinaikkan dan data dimasukkan ke dalam array.

Selanjutnya, method pop() di program saya tulis pop_3019() digunakan untuk mengambil sekaligus menghapus elemen teratas dari stack. Method ini akan mengecek kondisi *underflow* apabila stack kosong. Jika tidak kosong, maka elemen teratas akan diambil dan nilai top_3019 akan diturunkan.

Method peek_3019() berfungsi untuk melihat elemen teratas tanpa menghapusnya, sedangkan method print_3019() digunakan untuk menampilkan seluruh isi stack dari elemen paling atas hingga ke bawah.

3) Program ini tidak akan bisa dijalankan (run) langsung karena di dalam class ini tidak ada method main, jadi program ini hanya dijadikan class object yang nanti akan di jalankan oleh class driver/client nya.

2.1.2 Membuat kelas kedua (stackArrayDriver)

1) Klik kanan package pekan2 dan pilih "New", pilih class. Lalu buat class baru dengan nama "stackArrayDriver _2511533019."

2) Masukkan syntax seperti berikut.

```
1) package pekan3_2511533019;
2)
3) public class stackArrayDriver_2511533019 {
4)
5)    public static void main(String[] args) {
6)          // TODO Auto-generated method stub
7)          stackArray_2511533019 s = new
   stackArray_2511533019();
8)          s.push_3019(10);
9)          s.push_3019(20);
10)         s.push_3019(30);
11)         System.out.println(s.pop_3019()+ "
   dikeluarkan dari stack");
12)         System.out.println("elemen teratas adalah :
   "+s.peek_3019());
13)         System.out.print("elemen pada stack : ");
14)         s.print_3019();
15)
16)   }
17)
18) }
```

Program kedua yaitu *stackArrayDriver_2511533019* merupakan class yang digunakan untuk menguji implementasi struktur data stack yang telah dibuat pada kelas sebelumnya, yaitu stackArray_2511533019. Pada method main, program diawali dengan membuat sebuah objek stack bernama s. Selanjutnya, dilakukan operasi push untuk menambahkan beberapa elemen ke dalam stack, yaitu nilai 10, 20, dan 30. Sesuai dengan prinsip Last In First Out (LIFO), elemen terakhir yang dimasukkan akan berada di posisi teratas.

Setelah itu, program melakukan operasi pop untuk menghapus dan mengambil elemen teratas dari stack, yang kemudian ditampilkan ke layar. Berikutnya, method peek digunakan untuk melihat elemen yang berada di posisi paling atas setelah proses pop, tanpa menghapusnya dari stack. Terakhir, method print digunakan untuk menampilkan seluruh isi stack yang tersisa, dimulai dari elemen teratas hingga elemen terbawah.

Dengan adanya kelas driver ini, dapat dibuktikan bahwa operasi-operasi dasar pada stack seperti push, pop, peek, dan print telah berjalan dengan baik sesuai dengan konsep stack, sehingga mempermudah dalam memahami cara kerja struktur data tersebut melalui implementasi langsung.

3) Run program hingga nanti keluar output seperti berikut.

```
Console
<terminated> stackArrayDriver_2511533019
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
elemen teratas adalah : 20
elemen pada stack :  20 10
```

2.1.3 Membuat program ketiga (StackPostfix)

1) Klik kanan package pekan2 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “StackPostfix_2511533019.”

StackPostfix_2511533019.java

2) Lalu buat/masukkan syntax seperti berikut.

```
1)  package pekan3_2511533019;
2)
3)  import java.util.Scanner;
4)  import java.util.Stack;
5)
6)  public class StackPostfix_2511533019 {
7)    public static int postfixEvaluate_3019(String expression)
    {
8)          Stack<Integer> s_3019 = new Stack<Integer>();
9)          Scanner input_3019 = new Scanner(expression);
10)         while(input_3019.hasNext()) {
11)                 if (input_3019.hasNextInt()) { //an operan
    (integer)
12)                     s_3019.push(input_3019.nextInt());
```

```
13)         } else {
14)                 String operator_3019 = input_3019.next();
15)                 int operand2_3019 =s_3019.pop();
16)                 int operand1_3019 =s_3019.pop();
17)                 if (operator_3019.equals("+")) {
18)                         s_3019.push(operand1_3019 +
    operand2_3019);
19)                 }else if (operator_3019.equals("-")) {
20)                         s_3019.push(operand1_3019 -
    operand2_3019);
21)                 }else if (operator_3019.equals("*")) {
22)                         s_3019.push(operand1_3019 *
    operand2_3019);
23)                 }else {
24)                         s_3019.push(operand1_3019 /
    operand2_3019);
25)                 }
26)         }
27)   }
28)   input_3019.close();
29)   return s_3019.pop();
30)   }
31)   public static void main(String[] args) {
32)         // TODO Auto-generated method stub
33)         System.out.println("hasil postfix=
    "+postfixEvaluate_3019(" 5 2 4 * + 7 -"));
34)   }
35)
36) }
```

Program ketiga yaitu *StackPostfix_2511533019* merupakan program yang berfungsi untuk melakukan evaluasi ekspresi **postfix** dengan memanfaatkan struktur data stack.

Pada method postfixEvaluate_3019, dibuat sebuah objek stack dari Stack<Integer> yang berfungsi untuk menyimpan operand sementara selama proses perhitungan. Selain itu, digunakan objek Scanner untuk membaca setiap elemen dari string ekspresi postfix yang diberikan.

Proses evaluasi dilakukan dengan perulangan while selama masih terdapat input. Jika token yang dibaca merupakan bilangan bulat (*operand*), maka nilai tersebut akan langsung dimasukkan ke dalam stack menggunakan method push. Namun, jika token berupa operator, maka program akan mengambil dua elemen teratas dari stack menggunakan pop, yaitu sebagai operand pertama dan kedua.

Selanjutnya dilakukan operasi aritmatika sesuai operator yang ditemukan, seperti penjumlahan, pengurangan, perkalian, atau pembagian, dan hasilnya akan dimasukkan kembali ke dalam stack.

Setelah seluruh ekspresi selesai diproses, nilai terakhir yang tersisa di dalam stack merupakan hasil akhir dari evaluasi postfix, yang kemudian dikembalikan oleh fungsi. Pada method main, program memanggil fungsi tersebut dengan contoh ekspresi "5 2 4 * + 7 -", sehingga hasil perhitungan akan ditampilkan ke layar. Program ini menunjukkan bagaimana stack digunakan secara efektif untuk memproses ekspresi matematika tanpa memerlukan tanda kurung.

3) Run program hingga keluar output seperti berikut, apabila hasil perhitungannya tidak benar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.

```
Console
<terminated> StackPostfix_2511533019 [
hasil postfix= 6
```

2.1.4 Membuat program keempat (NilaiMaksimum)

1) Klik kanan package pekan3 dan pilih “New”, pilih class. Lalu buat nama “NilaiMaksimum_2511533019”.

```
> pekan3_2511533019
  NilaiMaksimum_2511533019.java
```

2) Masukkan syntax seperti berikut.

```
1)  package pekan3_2511533019;
2)  import java.util.Stack;
3)
4)  public class NilaiMaksimum_2511533019 {
5)   public static int max_3019(Stack<Integer> s_3019) {
6)          Stack<Integer> backup = new Stack<Integer>();
7)          int maxValue_3019= s_3019.pop();
8)          backup.push(maxValue_3019);
9)          while (!s_3019.isEmpty()) {
10)                 int next_3019= s_3019.pop();
11)                 backup.push(next_3019);
12)                 maxValue_3019 = Math.max(maxValue_3019,
    next_3019);
13)         }
14)         while (!backup.isEmpty()) {
15)                 s_3019.push(backup.pop());
16)         }
17)         return maxValue_3019;
18) }
19) public static void main(String[] args) {
20)         // TODO Auto-generated method stub
21)         Stack<Integer> s_3019= new Stack<Integer>();
22)         s_3019.push(70);
23)         s_3019.push(12);
24)         s_3019.push(20);
```

```
        System.out.println("isi stack "+s_3019);
        System.out.println("Stack teratas "+s_3019.peek());
        System.out.println("Nilai Maksimum "+max_3019(s_3019));
 }

}
```

Program keempat yaitu *DaftarKataDriver_2511533019* juga merupakan program driver yang berfungsi untuk mencari nilai maksimum dari elemen-elemen yang ada dalam sebuah stack. Pada method max_3019, digunakan dua buah stack, yaitu stack utama (s_3019) dan stack bantu (backup). Proses diawali dengan mengambil elemen teratas dari stack utama sebagai nilai maksimum sementara, kemudian elemen tersebut juga disimpan ke dalam stack backup.

Selanjutnya, program melakukan perulangan selama stack utama belum kosong. Pada setiap iterasi, elemen di-pop dari stack utama, kemudian dibandingkan dengan nilai maksimum sementara menggunakan fungsi Math.max. Jika ditemukan nilai yang lebih besar, maka nilai maksimum akan diperbarui. Setiap elemen yang diambil juga disimpan ke dalam stack backup agar data tidak hilang.

Setelah semua elemen selesai diproses, dilakukan proses pengembalian data dari stack backup ke stack utama, sehingga isi stack tetap seperti semula. Nilai maksimum yang telah diperoleh kemudian dikembalikan sebagai hasil fungsi. Pada method main, program membuat stack dan mengisi beberapa nilai, kemudian menampilkan isi stack, elemen teratas, serta nilai maksimum yang diperoleh.

Program ini menunjukkan bahwa pencarian nilai maksimum dapat dilakukan dengan memanfaatkan stack tanpa menghilangkan data aslinya, sehingga struktur data tetap terjaga.

3) Run program hingga keluar output seperti berikut, apabila error atau hasil nilai max tidak sesuai cek kembali syntax dan perbaiki kesalahan.

```
Console
<terminated> NilaiMaksimum_2511533019 [Java Application] C:\Use
isi stack [70, 12, 20]
Stack teratas 20
Nilai Maksimum 70
```

2.1.5 Membuat program keempat (SiswaStack)

1) Klik kanan package pekan3 dan pilih "New", pilih class. Lalu buat nama "SiswaStack_2511533019".
2) Ketik/buatlah syntax seperti berikut.

```java
package pekan3_2511533019;

import java.util.ArrayList;

class Siswa_2511533019 {
  String nama;
  int nim;

  public Siswa_2511533019(String nama, int nim) {
          this.nama = nama;
          this.nim = nim;
  }

  @Override
  public String toString() {
          return "Nim: " + nim + ", Nama: "+ nama;
  }
}
public class SiswaStack_2511533019 {
  private ArrayList<Siswa_2511533019> stack;

  public SiswaStack_2511533019() {
          stack = new ArrayList<>();
  }
  public void push_3019(Siswa_2511533019 mhs) {
          stack.add(mhs);
  }
  public Siswa_2511533019 pop_3019() {
          if (!isEmpty_3019()) {
                  return stack.remove(stack.size()- 1);
          }
          return null;
  }

  public Siswa_2511533019 peek_3019() {
          if (!isEmpty_3019()) {
                  return stack.get(stack.size()-1);
          }
          return null;
```

```
40)   }
41)   public boolean isEmpty_3019() {
42)           return stack.isEmpty();
43)   }
44)
45)   public void tampilkanSiswa_3019() {
46)           for(int i= stack.size()-1; i >= 0; i--) {
47)                   System.out.println(stack.get(i));
48)           }
49)   }
50)   public static void main(String[] args) {
51)           SiswaStack_2511533019 studentStack_3019 = new
    SiswaStack_2511533019();
52)
53)           Siswa_2511533019 mhs1 = new Siswa_2511533019("Ali",
    1);
54)           Siswa_2511533019 mhs2 = new
    Siswa_2511533019("Boby", 2);
55)           Siswa_2511533019 mhs3 = new
    Siswa_2511533019("Charles", 3);
56)
57)           studentStack_3019.push_3019(mhs1);
58)           studentStack_3019.push_3019(mhs2);
59)           studentStack_3019.push_3019(mhs3);
60)
61)           System.out.println("Siswa di dalam stack: ");
62)           studentStack_3019.tampilkanSiswa_3019();
63)
64)           System.out.println("siswa teratas "+
    studentStack_3019.peek_3019());
65)           System.out.println("mengeluarkan siswa teratas dari
    stack : "+studentStack_3019.pop_3019());
66)           System.out.println("daftar siswa setelah di pop :
    ");
67)           studentStack_3019.tampilkanSiswa_3019();
68)   }
69) }
```

Program SiswaStack_2511533019 ini digunakan untuk mengimplementasikan struktur data stack (tumpukan) yang berisi data siswa. Program ini berfungsi untuk menyimpan, menampilkan, serta mengelola data siswa menggunakan konsep LIFO (Last In First Out), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar. Dengan program ini, kita bisa melakukan operasi dasar stack seperti menambahkan data siswa, melihat data paling atas, menghapus data paling atas, dan menampilkan seluruh isi stack.

Program ini terdiri dari dua class utama, yaitu class Siswa_2511533019 dan class SiswaStack_2511533019. Class Siswa_2511533019 digunakan untuk merepresentasikan data siswa yang terdiri dari atribut nama dan nim, serta dilengkapi constructor

untuk mengisi data dan method toString() untuk menampilkan data siswa dalam bentuk yang rapi. Sedangkan class SiswaStack_2511533019 digunakan untuk mengatur struktur stack dengan bantuan ArrayList sebagai tempat penyimpanan data siswa.

Di dalam class stack ini terdapat beberapa method penting. Method push_3019() digunakan untuk menambahkan data siswa ke dalam stack. Method pop_3019() digunakan untuk menghapus data paling atas dari stack sekaligus mengembalikannya. Method peek_3019() digunakan untuk melihat data yang berada di posisi paling atas tanpa menghapusnya. Method isEmpty_3019() digunakan untuk mengecek apakah stack kosong atau tidak, sedangkan method tampilkanSiswa_3019() digunakan untuk menampilkan seluruh data siswa dari atas ke bawah sesuai konsep stack.

Pada bagian main program, dibuat beberapa objek siswa yaitu Ali, Boby, dan Charles, kemudian dimasukkan ke dalam stack menggunakan method push. Setelah itu program menampilkan isi stack, lalu menampilkan data paling atas menggunakan peek, kemudian menghapus data paling atas menggunakan pop. Terakhir, program kembali menampilkan isi stack setelah terjadi perubahan data.

3) Run program hingga console menampilkan output seperti berikut.

```
Console
<terminated> SiswaStack_2511533019 [Java Application] C:\Users\Filzi Jelila\.p2\pool\plug
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack : Nim: 3, Nama: Charles
daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

2.2.2 Membuat Kelas ADT Website

1) Klik kanan Package pekan3 dan pilih “New”, pilih class. Lalu buat nama “Website_2511533019”.
2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package pekan3_2511533019;
2)
3) public class Website_2511533019 {
4)       private String Judul_3019;
5)       private String URL_3019;
6)
7)       //konstruktor
8)       public Website_2511533019(String Judul,
   String URL) {
9)             this.Judul_3019 = Judul;
10)            this.URL_3019 = URL;
11)      }
12)
13)      //getter
14)      public String getJudul_3019(){
15)            return Judul_3019;
16)      }
17)      public String getURL_3019() {
18)            return URL_3019;
19)      }
20)
21)      //setter
22)      public void setJudul_3019(String Judul) {
23)            this.Judul_3019= Judul;
24)      }
25)      public void setURL_3019(String URL) {
26)            this.URL_3019 = URL;
27)      }
28)}
```

Kelas Website_2511533019 merupakan kelas ADT yang digunakan untuk merepresentasikan data sebuah website yang terdiri dari atribut judul dan URL. Dua atribut tersebut saya simpan dengan bertipe data String, dan variabel yaitu Judul_3019 dan URL_3019. Selain itu, kelas ini dilengkapi dengan konstructor untuk menginisialisasi nilai awal objek saat dibuat, serta method getter dan setter yang berfungsi untuk mengambil dan mengubah nilai atribut. Dengan adanya kelas ini, setiap data website dapat disimpan sebagai objek yang nantinya akan dikelola dalam struktur data stack.

3) Program ini tidak akan bisa di run secara langsung karna dia merupakan kelas object yang membutuhkan kelas baru(client/drivernya) agar bisa menampilkan suatu output.

2.2.3 Membuat Driver (Browser_2511533019)

1) Klik kanan Package pekan3 dan pilih "New", pilih class. Lalu buat nama "Browser_2511533019".
2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1)  package pekan3_2511533019;
2)
3)  import java.util.Scanner;
4)  import java.util.Stack;
5)
6)  public class Browser_2511533019 {
7)        static Stack<Website_2511533019> history = new
    Stack<>();
8)
9)        public static void main(String[] args) {
10)               // TODO Auto-generated method stub
11)               Scanner input = new Scanner(System.in);
12)         int pilihan;
13)         do {
14)             System.out.println("\n=== Browser History NIM
    : 2511533019 ===");
15)             System.out.println("1.Kunjungi Website");
16)             System.out.println("2.Tombol Back");
17)             System.out.println("3.Lihat Halaman Aktif");
18)             System.out.println("4.Cek History");
19)             System.out.println("5.Keluar");
20)             System.out.print("Pilihan: ");
21)             pilihan = input.nextInt();
22)             input.nextLine();
23)
24)          switch (pilihan) {
25)               case 1:
```

```
                    System.out.print("Masukkan Judul: ");
                    String judul = input.nextLine();
                    System.out.println("Masukkan URL: ");
                    String url = input.nextLine();

                    history.push(new Website_2511533019(judul, url));
                    System.out.println("Berhasil mengunjungi halaman web!");
                    break;

            case 2:
                    if (!history.isEmpty()) {
                            Website_2511533019 removed = history.pop();
                            System.out.println("back from : " + removed.getJudul_3019());
                    } else {
                            System.out.println("History kosong!!");
                    }break;

            case 3:
                    if (!history.isEmpty()) {
                            Website_2511533019 top = history.peek();
                            System.out.println("Halaman saat ini:");
                            System.out.println("Judul: " + top.getJudul_3019());
                            System.out.println("URL: " + top.getURL_3019());
                    } else {
                            System.out.println("Kosong!");
                    }break;

            case 4:
                    System.out.println("Jumlah history: " + history.size());
                    break;
            }
        }while(pilihan !=5);
    }

}
```

3) Kelas Browser_2511533019 merupakan kelas driver yang digunakan untuk menjalankan simulasi browser history. Pada kelas ini terlihat bahwa saya sudah menggunakan struktur data Stack dari java.util.Stack untuk menyimpan objek website, yang dimana setiap data yang dimasukkan akan mengikuti konsep LIFO (Last In First Out). Stack tersebut disimpan dalam variabel history dengan tipe Stack<Website_2511533019>. Dalam kelas ini, saya juga mengimplementasikan beberapa fitur

utama seperti push untuk menambahkan website yang dikunjungi, pop untuk menghapus halaman terakhir saat tombol back ditekan, serta peek untuk melihat halaman yang sedang aktif tanpa menghapusnya dari stack. Selain itu, saya juga menggunakan method isEmpty() untuk menangani kondisi ketika stack kosong agar program tidak mengalami error saat melakukan operasi pop atau peek. Program ini dibuat interaktif dengan menggunakan perulangan do-while dan input dari user melalui Scanner, sehingga pengguna dapat memilih menu yang tersedia seperti mengunjungi website, kembali ke halaman sebelumnya, melihat halaman aktif, serta mengecek jumlah history yang tersimpan. Sesuai dengan kriteria tugas.

4) Run program, dan inputkan sesuai perintah. Contoh output dengan berbagai input yang telah saya masukkan :

```
Console
<terminated> Browser_2511533019 [Java Application] C:\Users\Filzi Jelila\.p2\pool\plu

=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 1
Masukkan Judul: ilearn unand
Masukkan URL:
https://it.ilearn.unand.ac.id/
Berhasil mengunjungi halaman web!

=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 2
back from : ilearn unand

=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 3
Kosong!
```

```
Console
<terminated> Browser_2511533019 [Java Application] C:\Users\Filzi Jelila\.p2\pool\plu
=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 2
History kosong!!

=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 4
Jumlah history: 0

=== Browser History NIM : 2511533019 ===
1.Kunjungi Website
2.Tombol Back
3.Lihat Halaman Aktif
4.Cek History
5.Keluar
Pilihan: 5
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan struktur data linear yang menerapkan prinsip LIFO (Last In First Out), yaitu elemen yang terakhir masuk akan menjadi elemen yang pertama keluar. Jika dibandingkan dengan ArrayList, stack memiliki perbedaan utama pada cara pengelolaan datanya, di mana ArrayList bersifat lebih fleksibel karena data dapat diakses, ditambah, dihapus, dan diubah dari berbagai posisi menggunakan indeks, sedangkan stack hanya memungkinkan akses pada elemen paling atas saja sehingga lebih terbatas namun terstruktur sesuai aturan LIFO.

Implementasi stack pada berbagai program menunjukkan bahwa struktur data ini sangat berguna dalam pengelolaan data secara terurut, seperti pada operasi push, pop, peek, dan pengecekan data. Selain itu, penggunaan stack juga dapat diterapkan dalam berbagai kasus seperti evaluasi ekspresi postfix, pencarian nilai maksimum, serta pengelolaan data objek seperti data siswa.

Dengan demikian, praktikum ini berhasil memberikan pemahaman yang lebih jelas mengenai konsep stack, cara implementasinya dalam pemrograman Java, perbedaannya dengan ArrayList, serta penerapannya dalam berbagai permasalahan komputasi secara dinamis dan terstruktur.

DAFTAR PUSTAKA

[1] Wahyudi, "*Stack*" PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2026.

[2] Oracle, "Stack Class in Java," Oracle Documentation, 2024. [Online]. Available: https://docs.oracle.com/javase/ [Diakses: Apr. 10, 2026].